Variables and Strings

Introduction to computer programming

Management and Artificial Intelligence

Variables = a Name for Values

The equal sign `=` serves for **assigning** a variable name to a value:

```
>>> pi = 3.14
```

where `pi` is the *variable name* and `3.14` is the *value*.

> **Recall**: The equal sign `=` serves for assignments. The equality check reads as `==`.

Retrieve the value associated to a variable by querying the name on the prompt:

```
>>> pi      # input
3.14        # output
```

- All values are stored in the computer memory
- The assignment operation binds a name to a value

Variable names:

- must be meaningful
- can be as long as you like
- can contain both letters and numbers
- cannot start with a number
- can contain underscores (i.e., `_`)
- convention: use only lower letters
- Python-reserved keywords are not allowed:

`False`	`class`	`finally`	`is`	`return`
`None`	`continue`	`for`	`lambda`	`try`
`True`	`def`	`from`	`nonlocal`	`while`
`and`	`del`	`global`	`not`	`with`
`as`	`elif`	`if`	`or`	`yield`
`assert`	`else`	`import`	`pass`	`break`
`except`	`in`	`raise`		

Why variable names?

- reuse **names** instead of **values** throughout the code
- store (temporarily!) values and results
- easier to change the code later

Examples:

```
>>> pi=3.14
>>> radius=2.2
>>> # calculate the area        <-- this is a comment!
>>> area = pi * (radius**2)
>>> # calculate the diameter
>>> diameter = radius*2
```

Syntax of assignments:

- rightmost term of assignment: expression that yields a value
- leftmost term of assignment: variable name

Variables can be **changed** (or **re-binded**) using new assignments on the same variable name.

Warning: The old value might still be in the computer memory, but you lose the handle for it.

```
>>> pi=3.14
>>> radius=2.2
>>> area = pi * (radius**2)
>>> radius = radius+1
```

Hint: `radius = radius+1` can be shortcut to `radius+=1`.

Note: `area` does not change its value.

Dynamic Typing

Python supports **dynamic typing**.

- variables can hold values of any type
- variables can hold different types throughout the code

The following is perfectly valid in Python:

```
>>> x=3          # x now contains an int
>>> ...
>>> x=9.53642    # x now contains a float
```

Recall: You can always use `type(x)` to find the type of variable `x`.

You can use `type()` for comparison and conversion

```
>>> x=3
>>> type(x)==int
True
>>> x=float(x)
>>> type(x)==float
True
```

Strings: a Warmup

Strings are objects that can host letters, numbers, spaces, special characters. Strings can be created by enclosing stuff in quotation marks or single quotes:

```
>>> hi='hello there' # or "hello there"
```

You can **concatenate** strings using the `+` operator:

```
>>> name='alessio'
>>> greeting=hi + name
>>> greeting
'hello therealessio'
>>> greeting=hi + " " + name
>>> greeting
'hello there alessio'
```

You can **replicate** strings using the `*` operator:

```
>>> name='alessio'
>>> silly=name*3
>>> silly
'alessioalessioalessio'
```

Single quotes or double quotes can be (in principle) used interchangeably. For example:

```
>>> s1 = 'hello students'
>>> s2 = "hello students"
```

yield two identical strings.

Problem: What if my string contains itself a single quote (e.g., *Let's go*)?

If we do something like

```
>>> s = 'Let's go'
SyntaxError: invalid syntax
```

we have an unterminated string syntax error.

Solution: Wrap your string with single quotes between double quotes!

```
>>> s = "Let's go"
```

(Opposite) problem: What if my string contains itself a double quote (e.g., *I said "Hello"*)?

If we do something like

```
>>> s = "I said "Hello""
SyntaxError: invalid syntax
```

we have another syntax error, equivalent to the previous one.

Solution: Wrap your string with double quotes between single quotes!

```
>>> s = 'I said "Hello"'
```

(Yet another) problem: What if my string contains both single quotes and double quotes (e.g., *I said "What's going on"*)?

We cannot do any of the following

```
>>> s = "I said "What's going on""
SyntaxError: invalid syntax
>>> s = 'I said "What's going on"'
SyntaxError: invalid syntax
```

because of syntax errors, equivalent to the previous ones.

Solution: Wrap your string between triple single quotes!

```
>>> s = '''I said "What's going on"'''
```

Escape Sequences

We can solve the above problem also thanks to **escape sequences**.

Escape sequences are brief sequences of characters that start with a backslash and are interpreted differently.

There exist escape sequences to display single quotes (`\'`) and double quotes (`\"`) ... and to display the backslash itself (`\\`):

- if we 'shield' single or double quotes with a backslash, they are interpreted as characters rather than string delimiters;
- if we 'shield' the backslash with a backslash, it is interpreted as character rather than 'this the beginning of an escape sequence'.

Let's try the following:

```
>>> s1 = 'I said "What\'s going on"'
>>> s2 = "I said \"What's going on\""
>>> s3 = "Hello\\Students"
```

There exist escape sequences also for string formatting.

Let's see the most important ones:

- new line `\n`, which changes the line and takes the cursor to the beginning of a new line
- tab space `\t`, which leaves a horizontal tab space
- carriage return `\r`, which moves the cursor back to the beginning of the line
- alarm `\a`, which plays the system alarm (bell)
- form feed `\v` or `\f`, which breaks the line

Let's see them in action:

```
>>> s1 = 'Hello\nStudents'
>>> print(s1)
Hello
Students
>>> s2 = 'Hello\tStudents'
>>> print(s2)
Hello   Students
>>> s3 = 'Hello\rStudents'
>>> print(s3)
Students
>>> print('\a')
```

Input/Output

- Show some stuff to the command line
- Universal (and very flexible) keyword: `print()`

Examples:

```
>>> print('apple', 'banana') # two input arguments
apple banana
>>> print('apple' + 'banana') # one input argument
applebanana
```

By default, multiple input arguments are separated by space ( ) and the line terminates with a newline ('`\n`'). You can change that via `sep` and `end`.

Example:

```
>>> a=5
>>> print("a=", a, 10, "bye", sep='|', end='\n\n\n')
a=|5|10|bye


>>>
```

Hence, there are two strategies to print something on screen:

1. pass multiple arguments to `print()` and it automatically takes care of printing strings, numbers and different data types
2. build your own string and just `print()` it.

The second case deserves further observations.

What happens if we run the following code?

```
>>> age=5
>>> name='Eric'
>>> print("My name is "+name+" and I'm "+str(age)+" years old.")
My name is Eric and I'm 5 years old.
```

It will error because we cannot concatenate (i.e., `+`) strings and numbers together. Hence, we must convert `age` to string (i.e., using `str()`).

String Formatting

Luckily, Python has several string formatting shortcuts that allow us to create strings

- without manual concatenation and casting
- in a very elegant and clean way.

Let's see the old-school way (C-like).

What happens if we run the following code?

```
>>> n1 = 5
>>> n2 = 1234.56789
>>> print("The value of n1 is %d" % n1)
The value of n1 is 5
>>> print("The value of n2 is %d" % n2)
The value of n2 is 1234
>>> print("The value of n1 is %f" % n1)
The value of n1 is 5.000000
>>> print("The value of n2 is %f" % n2)
The value of n2 is 1234.567890
```

In the string we are building, we can see that:

- the percentage sign marks the place where the value has to be inserted
- `%` is followed by the conversion format, i.e.:
    - `%d` stands for decimal (i.e., integer)
    - `%f` stands for floating point
- the format automatically performs casting
    - `n1` is an integer, but it renders as float if we use `%f`
    - `n2` is a float, but it renders as integer if we use `%d`

Outside the string we are building, we note that:

- `%` is followed by the variable name (or value) we want to plug inside the string

Note: We can personalize `%f` by specifying the number of digits before and after the decimal separator. That is, `%$x$.$y$f` yields a floating point number with x digits before and digits after the comma. can be omitted.

We can build strings with an arbitrary number of placeholders.

Let's run the following code

```
>>> name = "Eric"
>>> age = 5
>>> print("My name is %s and I'm %d years old." %(name, age))
My name is Eric and I'm 5 years old.
```

We note that:

- we have two placeholders inside the string
    - `%s` stands for strings
    - `%d` follows from previous slide
- outside the string, we have our variables/values collected in round brackets (That's a *tuple*, we will see tuples later in the course.)
- the order of the placeholders must match the ones in the tuple

Note: `%s` can be used for strings and for all objects that support a string conversion, e.g., numbers.

Input: `input()`

- Ask the user for some input via keyboard
- Keyword: `input('...')`

How does it work?

1. prints whatever is in the quotes
2. user types something and press Enter
3. binds the value to a variable

Example:

```
>>> text=input('Type something: ')
Type something: hello
>>> print(text*3)
hellohellohello
```

Warning: `input()` always returns a string. If you have to play with numbers, you must cast.

Example:

```
>>> num=input('Enter a number: ')
Enter a number: 5
>>> num
'5'
>>> type(num)
<class 'str'>
>>> num = int(num)
>>> type(num)
<class 'int'>
```

Some Recaps

- **Type:** `int`
  - values: integers
  - operations: `+`, `-`, `/`, `//`, `*`, `%`, `**`, ...
- **Type:** `float`
  - values: real numbers
  - operations: `+`, `-`, `/`, `//`, `*`, `%`, `**`, ...
- **Type:** `bool`
  - values: `True` and `False`
  - operations: `or`, `and`, `not`
- **Type:** `str`
  - values: string literals
  - operations: `+`, `*`, ...

Note: We will see more (non-scalar) data types later in the course.

Interactive vs. Script

What happens if you type this code in the interpreter or write a script and execute it?

```
>>> miles = 11.41
>>> miles
11.41
>>> miles * 2
22.82
```

Interactive	Script
Assignments are silent	Assignments are silent
Variable query returns its content	Variable query is silent
Expressions are returned in real-time	Expressions are silent

Note: If you want your script to show something, you must use `print()`.

Expressions vs. Statements

- **Expressions: Represent** something
    - Python *evaluates* expressions
    - end-result is a value

  Examples:

```
>>> 2.3          # a plain literal
2.3
>>> (7*3+1)-4    # 4 literals and some operators
18
```

- **Statements: Do** something
    - Python *executes* statements
    - no need to return a value

  Examples:

```
>>> print("Hi there!")
Hi there!
>>> import sys
```